

STRUCTURED ABSTRACT

<https://www.engineersaustralia.org.au/AAFE2013/>

Enhancing Code Comprehension through Prompt Engineering and Large Language Models: A Practical Web-Based Tool

CONTEXT

The integration of Artificial Intelligence in programming education is rapidly advancing, offering personalized support and lowering entry barriers for novice developers. Large Language Models (LLMs) have emerged as intelligent assistants capable of generating, analysing, and optimizing source code [1]. Effective interaction with these models, however, requires skills in prompt engineering – a growing research field focused on refining user input for more accurate responses [2].

PURPOSE OR GOAL

This study explores prompt engineering techniques and their application to code analysis via LLMs. The aim was to develop an educational tool that simplifies code interaction for students and novice programmers, allowing them to receive high-quality feedback without crafting prompts themselves. The tool bridges the gap between user intention and LLM output by offering pre-designed, optimized prompts.

APPROACH

A web-based application was built utilizing React.js, communicating directly with the OpenRouter API in a client-API architecture to handle queries and responses. Users can input code, select among multiple LLMs, and choose from ten predefined prompts. Each prompt was based on established strategies like zero-shot, few-shot, and chain-of-thought, optimized for code analysis. A standardized prompt structure – featuring a fixed “role”, dynamic “task” and “requirements” sections – was applied across all prompts, balancing consistency with flexibility according to each use case.

ACTUAL OR ANTICIPATED OUTCOMES

Although users cannot edit the prompts, they can observe how different types affect LLM responses in various coding scenarios, gaining intuitive understanding of prompt influence. The tool is expected to support programming education through guided code explanations and applicable suggestions. The initial tests of the proposed application show that this project may prove a useful support for users with limited technical expertise to interpret and improve code using LLMs’ output.

CONCLUSIONS/RECOMMENDATIONS/SUMMARY

When properly applied, prompt engineering enhances the educational value of LLMs in programming contexts. The tool provides low-barrier access to advanced AI support and underscores the role of structured prompt design in learning environments. Future work includes empirical testing with students and the addition of features that allow users to view, adjust, and compare prompt structures based on their experience level.

REFERENCES

1. Wang, T., Zhou, N., & Chen, Z. Enhancing computer programming education with LLMs: A study on effective prompt engineering for Python code generation. arXiv. <https://doi.org/10.48550/arXiv.2407.05437> (2024).
2. Google Cloud: https://cloud.google.com/discover/what-is-prompt-engineering?utm_source=chagpt.com&hl=en, last accessed 2025/04/15.

KEYWORDS

Prompt Engineering, Code Analysis, Large Language Models, Programming Education.